Appendix A.2

1. struct thread (in threads/thread.h)

```
struct thread
  {
    /* Owned by thread.c. */
    tid_t tid;                          /* Thread identifier. */
    enum thread_status status;          /* Thread state. */
    char name[16];                      /* Name (for debugging purposes). */
    uint8_t *stack;                     /* Saved stack pointer. */
    int priority;                       /* Priority. */
    struct list_elem allelem;           /* List element for all threads list. */

    /* Shared between thread.c and synch.c. */
    struct list_elem elem;              /* List element. */

#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir;                  /* Page directory. */
#endif

    /* Owned by thread.c. */
    unsigned magic;                     /* Detects stack overflow. */
  };
```

tid_t tid; (int) 시스템에서 유일함.

int priority; 0~63 / 숫자가 높을 우선순위

struct list_elem allelem; 전체 스레드 리스트에 연결할 때 사용. thread_foreach() => 모든 스레드 반복

struct list_elem elem; doubly linked list에 넣음. ready_list (실행 준비된 스레드 목록) sema_down (세마포어에서 대기중인 스레드 목록)

uint32_t *pagedir; project 2 ~

unsigned magic; 항상 THREAD_MAGIC 값. → stack overflow 발생시 덮어쓰임. ASSERT(thread->magic == THREAD_MAGIC)

status

`enum thread_status status` [Member of `struct thread`]

The thread's state, one of the following:

`THREAD_RUNNING` [Thread State]

The thread is running. Exactly one thread is running at a given time. `thread_current()` returns the running thread.

`THREAD_READY` [Thread State]

The thread is ready to run, but it's not running right now. The thread could be selected to run the next time the scheduler is invoked. Ready threads are kept in a doubly linked list called `ready_list`.

`THREAD_BLOCKED` [Thread State]

The thread is waiting for something, e.g. a lock to become available, an interrupt to be invoked. The thread won't be scheduled again until it transitions to the `THREAD_READY` state with a call to `thread_unblock()`. This is most conveniently done indirectly, using one of the Pintos synchronization primitives that block and unblock threads automatically (see Section A.3 [Synchronization], page 66).

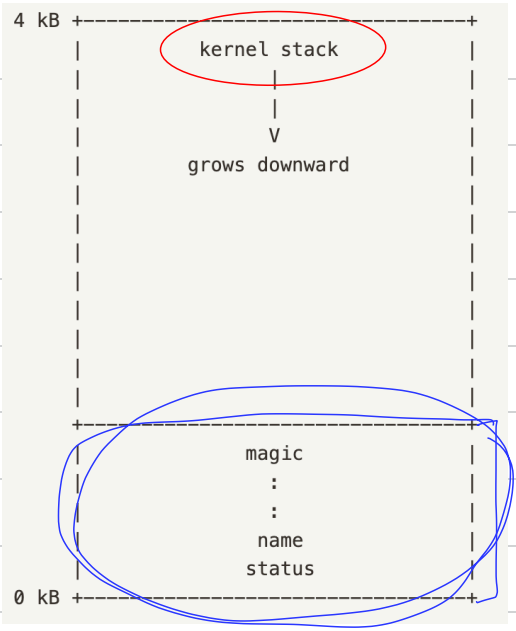
There is no *a priori* way to tell what a blocked thread is waiting for, but a backtrace can help (see Section E.4 [Backtraces], page 103).

`THREAD_DYING` [Thread State]

The thread will be destroyed by the scheduler after switching to the next thread.

하나의 스레드

→ 한페이지 (4KB) 메모리 사용



주의 : struct thread 크기 조심.

지역변수 a[1000] 말고 malloc()이나

palloc_get_page() 동적할당 사용.

너무 키우지 말자.

커널 스택 공간이 부족해짐.